Lightweight and Efficient DDoS Victim Detection in Programmable Data Planes

mingxue Ji[†], Xiong Wang*[†], Jing Ren[†], Rongping Lin[†], Sheng Wang[†], Shizhong Xu[†]
[†]University of Electronic Science and Technology of China, Chengdu, China

***Abstract*—Detecting Distributed Denial of Service (DDoS) attackss efficiently within resource-constrained programmable networks poses significant challenges due to limited memory and processing capabilities on hardware switches. This paper focuses on the crucial first stage of DDoS detection: lightweight victim detection directly in the resource-constrained data plane. We propose two novel, sketch-based mechanisms tailored for programmable switches: DualSketch and HashGuard. DualSketch employs two cascaded Count-Min (CM) Sketches with different bit widths and a carry mechanism, preceded by a Bloom filter to count unique source IPs per destination, achieving storage efficiency. HashGuard combines a Bloom filter and CM Sketch for preliminary filtering with a hash table to dynamically track suspicious high fan-in destination IPs, optimizing memory usage and reducing sketch update frequency. Theoretical error analyses are provided. Comprehensive evaluations using real network traffic (MAWI dataset) and comparison against the state-of-the-art scheme demonstrate that both DualSketch and HashGuard significantly reduce memory consumption while achieving higher detection accuracy (Precision, Recall, F1 Score) and lower estimation error. They attain high F1 scores (>0.90) even under severe memory constraints (e.g., 3MB), proving their effectiveness for practical deployment in programmable network environments.**

***Index Terms*—DDoS Detection, sketch, programmable network.**

I. Introduction

With the continuous evolution of the Internet, network traffic volumes have steadily increased while network environments have become increasingly complex and dynamic. The prevalence of anomalous traffic has risen significantly, highlighting the growing importance of anomaly detection mechanisms. Anomalous traffic encompasses various types, including DDoS attacks, super spreaders, port scanning [1], and worm propagation [2], with DDoS attacks constituting the majority of such anomalies. These attacks not only cause service disruptions but may also lead to serious consequences such as data breaches and resource exhaustion, posing substantial threats to network security. Traditional network devices, however, due to their fixed functionality and limited processing capabilities, typically rely on traffic sampling or mirroring for statistical analysis and detection, resulting in high rates of false positives and false negatives that significantly compromise network security.

In recent years, programmable networking technology has emerged as a critical milestone in network architecture development, with its core advantage being the ability to overcome the functional limitations of traditional network devices. Compared to earlier technologies like OpenFlow, modern programmable networks offer deeper programmability through Software-Defined Networking (SDN) and data plane programmability, enabling network operators to customize packet processing logic. In programmable switches, developers can define forwarding processes using high-level programming languages while executing lightweight computational tasks directly in the data plane without transferring data to the control plane. This architectural advantage facilitates efficient implementation of hash-based or sketch-based traffic measurement schemes, providing fine-grained visibility for all traffic and significantly enhancing anomaly detection capabilities. The combination of flexibility and computational power in programmable networks opens new avenues for sophisticated traffic management and security protection.

Although programmable networking technology has created new opportunities for implementing real-time DDoS anomaly detection in the network data plane—significantly reducing detection latency and system overhead while improving detection accuracy—programmable resources on hardware switches remain strictly limited. These resources include Match-Action Units (MAUs), Stateful Action Logic Units (SALUs), hash units, and memory components, all of which are highly constrained in practical deployments. For example, the widely used Intel Tofino ASIC is equipped with only 12 MAU pipeline stages, each with just 10 Mb of SRAM and 528 Ktrits of TCAM. More challenging still, these valuable resources must simultaneously support multiple network functions including routing, traffic engineering, security policies, and measurement statistics, requiring DDoS detection solutions to operate efficiently under severe resource constraints.

In traditional network architectures, DDoS attack detection primarily relies on traffic measurement tools such as Sflow and NetFlow to sample traffic, followed by algorithmic analysis of the sampled data to identify anomalies. However, in high-volume traffic scenarios, this approach incurs substantial computational overhead and suffers from sampling limitations, particularly for DDoS flows that transmit relatively few packets. The emergence of SDN and programmable networking technologies has made it possible to directly collect flow characteristics in the data plane. For instance, research [3] has proposed Sketch-based DDoS attack detection methods, but these approaches still face challenges related to high storage requirements and computational complexity, making them difficult to deploy in resource-constrained programmable network environments.

*Corresponding author, Email: wangxiong@uestc.edu.cn

To reduce the memory footprint and complexity of DDoS attack detection, the entire detection process can be divided into two critical phases: first identifying potential DDoS attack victims, then extracting features and making attack determinations only for traffic directed toward these victims. This paper focuses on the technical challenges of the first phase, aiming to design efficient, low-overhead victim detection mechanisms for programmable networks. By precisely filtering potential victims, we can significantly narrow the scope of traffic requiring in-depth analysis, substantially reducing feature extraction overhead and control plane computational burden within the limited resources of programmable switches, thereby improving overall detection efficiency and system scalability. To this end, we design two novel Sketch-based DDoS victim detection mechanisms: DualSketch and HashGuard. DualSketch innovatively cascades two CM Sketches with different bit widths and introduces a carry mechanism to achieve storage-efficient victim detection. HashGuard cleverly combines CM Sketch for preliminary filtering with hash tables for dynamic tracking of suspicious victims, optimizing memory utilization. Both methods achieve high-accuracy, low-overhead DDoS victim detection in resource-constrained programmable switch environments. The main contributions of this paper are summarized as follows:

(1) Addressing the challenge of DDoS attack victim detection, we propose two efficient programmable network-oriented mechanisms: DualSketch and HashGuard. Both achieve low-overhead and efficient potential DDoS victim detection through effective Sketch structure combinations.

(2) We provide rigorous theoretical error analysis of the proposed mechanisms, demonstrating detection accuracy guarantees under limited memory constraints and establishing a theoretical foundation for practical deployment.

(3) Comprehensive performance evaluations using real network traffic datasets show that our proposed mechanisms significantly reduce memory consumption while maintaining high detection precision compared to existing methods.

II. Related Work

Existing DDoS detection methods can be broadly classified into three categories: entropy-based, machine learning-based, and sketch-based approaches. Entropy-based methods [4]–[7] identify DDoS attacks by monitoring changes in information entropy, which reflects uncertainty in traffic characteristics like source and destination IP distributions. For instance, [4] detects attacks by analyzing entropy shifts in IP distributions, while [5] applies a similar technique tailored for BPFabric architecture. These methods often require complex analyses of multiple flow characteristics, posing challenges for direct data plane implementation. D. Ding et al. [6] introduced P4LogLog to estimate flow cardinality and normalize entropy for DDoS detection, but the computational complexity and subtle entropy changes post-normalization could lead to missed detections on programmable hardware.

Machine learning-based detection methods [8]–[11] first collect relevant flow features and then use classifiers for fine-grained discrimination. There are generally two approaches to implementing machine learning-based DDoS detection in programmable networks. One approach deploys both flow feature extraction and machine learning-based flow classification in the data plane [8], [10]; the other extracts flow features in the data plane and then sends these features to the control plane for classification using machine learning algorithms [8], [9], [11]. Compared to the first implementation method, the second approach reduces data plane implementation complexity and enables the use of more sophisticated models to improve DDoS flow classification accuracy. Machine learning-based DDoS attack detection methods can achieve fine-grained detection (e.g., identifying specific DDoS attack flows), but they require numerous features to be extracted. Even when DDoS flow identification is performed in the control plane, deploying relatively complex feature collection modules in the data plane presents a challenge for resource-constrained programmable networks.

In recent years, Sketches have become a popular tool for traffic measurement in programmable networks. These probabilistic data structures are designed to handle large-scale data streams, offering benefits such as low memory usage, high computational efficiency, and bounded error. Common Sketches include CM Sketch [12], CU Sketch [13], and Count Sketch [14]. However, using a single Sketch alone is often inadequate for effective DDoS attack detection; a combination of multiple data structures is typically necessary. For example, one study [15] combines CM Sketch with Hyperloglog to detect DDoS attacks by counting packets and cardinality for destination IPs, but the computational complexity of Hyperloglog makes it challenging to implement efficiently on programmable hardware. Another approach [3] integrates CM Sketch with Bitmap structures to detect DDoS attacks by counting distinct source IPs per destination IP, though it consumes considerable memory and is limited to single-switch detection, complicating cross-domain attack scenarios. Other studies [16], [17] also explore DDoS detection as a key use case for anomaly traffic identification, deploying specific hash functions and Sketch structures in the data plane to achieve high detection accuracy, though control plane involvement can impact timeliness and full exploitation of programmable network capabilities.

In summary, although current DDoS detection methods have advanced, many face challenges such as high computational complexity and large storage needs, making them hard to deploy on programmable hardware. Additionally, reliance on control plane involvement can hinder real-time performance. Thus, developing a lightweight, data plane-only solution for quickly identifying DDoS victims is crucial for efficient detection systems.

III. The Problem Description

In a DDoS attack, attackers use a botnet of numerous puppet hosts to send massive requests to the victim, causing multiple hosts to send packets to a single host, making the target host a potential DDoS attack victim. The ultimate

goal of DDoS attack detection is to identify attack flows. To reduce detection difficulty, DDoS attack detection can be divided into two stages: the first stage identifies potential victims; the second stage identifies DDoS attack flows from the streams directed at the victim. This paper focuses on the first stage, namely DDoS victim detection, aiming to design a lightweight and efficient pure data plane detection mechanism for programmable network scenarios.

Assume the switch receives a packet $pkt(IP_{src}, IP_{dst})$, where IP_{src} represents the IP address of the source host src, and IP_{dst} is the IP address of the destination host dst. We define N_{dst} as the number of distinct source hosts associated with the destination host dst, meaning there are N_{dst} different source hosts sending packets to the destination host dst. For convenience, this paper defines N_{dst} as the fan-in of node dst. We use $N_{dst}(t, t')$ to denote the Fan-in of the destination node dst within the time interval $[t, t']$. If the current destination host is under attack, then $N_{dst}(t, t')$ will inevitably exceed a certain threshold φ, i.e., $N_{dst} > \varphi$. In actual networks, the threshold φ can be obtained based on historical traffic characteristics or dynamically adjusted according to detection results.

IV. The DualSketch

A. The Data Structure

In DDoS victim detection, it is crucial to count the number of source IPs sending data to the same destination IP. Since a single flow $f(IP_{src}, IP_{dst})$ may send multiple packets, directly using a Sketch (e.g., CM Sketch) to count packet numbers can result in counts exceeding the number of source IPs. To address this issue, we first use a Bloom filter to filter out all but one packet of the same flow. Then, we use a CM Sketch to count the packets directed to each destination IP. Since each flow only triggers the CM Sketch insertion operation when its first packet arrives, we can obtain the number of source IPs sending packets to IP_{dst} by querying the CM Sketch using IP_{dst} as the key.

In real DDoS attack scenarios, victims constitute only a small fraction of nodes, so most nodes have a small fan-in. Hence, most counters in the CM Sketch may have small values. Allocating a large bit-width register for each counter in the CM Sketch to prevent overflow of a few counters would lead to resource wastage. To solve this problem, we employ a dual-layer CM Sketch cascading method (as shown in Fig. 1), where the first CM Sketch has small counter bit-width, and the second CM Sketch also has small counter bit-width, but

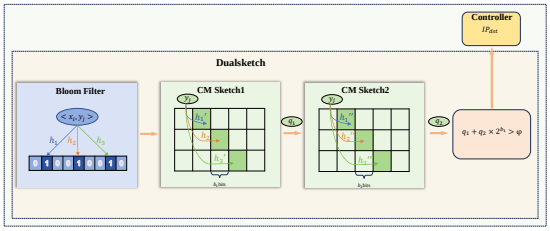


Fig. 1: The framework of Dualsketch

Algorithm 1 Dualsketch Workflow

for each packet $pkt(IP_{src}, IP_{dst})$ **do**
  Extract source and destination IP $\langle x_i, y_j \rangle$ from the packet
  if $\langle IP_{src}, IP_{dst} \rangle$ is not in Bloom Filter **then**
    Insert $\langle IP_{src}, IP_{dst} \rangle$ into Bloom Filter
    Query CM Sketch 1 using IP_{dst} as key and obtain query value q_1
    if $q_1 == 2^{b_1} - 1$ **then**
      Set the counter in CM Sketch 1 corresponding to IP_{dst} to 0
      Update CM Sketch 2 and obtain the query value q_2 for IP_{dst}
    else
      Update CM Sketch 1 and obtain the query value q_1 for IP_{dst}
    end if
    if $q_2 \times 2^{b_1} + q_1 > \varphi$ **then**
      Report IP_{dst} as a potential DDoS victim to the controller
    end if
  else
    Forward the packet to the switch output
  end if
end for

the number of counters in the second CM Sketch is much smaller than in the first. When a counter in the first CM Sketch overflows, it carries over to the second CM Sketch. Finally, when the fan-in of a certain destination IP exceeds ϕ, that destination IP is sent to the network controller for further processing.

B. The Workflow of Dual Sketch

The workflow of DualSketch is shown in **Algorithm 1**. When each packet arrives, the source IP and destination IP addresses $\langle IP_{src}, IP_{dst} \rangle$ are first extracted from the packet header. Then, the flow is checked in the Bloom Filter using $\langle IP_{src}, IP_{dst} \rangle$ as the key. If the result is true (indicating that this packet is the first packet of the flow), the packet is forwarded to the switch output. If the result is false, the Bloom Filter is updated, and the packet is forwarded to CM Sketch 1 for processing. When the packet reaches CM Sketch 1, it queries CM Sketch 1 using IP_{dst} as the key. If the query value $q_1 = 2^{b1} - 1$, it indicates a carry operation is needed, and the corresponding counter should be reset to zero while continuing to update the count in CM Sketch 2. If the CM Sketch 1 count is less than $2^{b1} - 1$, only the count in CM Sketch 1 is updated. During the update, we need to record the query values from both CM Sketch 1 and CM Sketch 2, denoted as q_1 and q_2, respectively. If $q_1 + q_2 \times 2^{b1} > \varphi$, then IP_{dst} is sent to the network controller for further processing.

C. The Error Analysis

Theorem 1: For a non-DDoS flow f_j, the probability P_j of it being identified as DDoS satisfies the following inequality:

$$P_j \leq \left[\frac{N(1-p)}{\omega(\varphi - n_j + n_j p)}\right]^d \tag{1}$$

where N represents the total number of flows entering the Bloom Filter, p represents the false positive rate of the Bloom Filter, ω represents the column width of the CM Sketch, d represents the number of rows in the CM Sketch, φ is the threshold, and n_j represents fan-in of the destination node of flow f_j.

Proof: Suppose the stream entering the Bloom Filter contains multiple non-DDoS flows f_j. Due to false positives in the Bloom Filter, assume it already contains N flows; then the false positive rate is:

$$p = \left[1 - \left(1 - \tfrac{1}{m}\right)^{kN}\right]^k, \tag{2}$$

where k is the number of hash functions and m is the number of bits in the Bloom Filter. Treating all flows as having the same false positive probability p, after filtering, the fan-in of flow f_j becomes:

$$n_j^b = n_j\,(1-p). \tag{3}$$

By the Probabilistic Error Bound for CM Sketch, if $\hat{n}_j$ is the estimate of n_j^b and $X_j = \hat{n}_j - n_j^b$, then:

$$E[X_j] = \frac{N\,(1-p)}{\omega}, \tag{4}$$

where ω is the column width of the CM Sketch.

According to Markov's inequality:

$$P\big(\hat{n}_j > \varphi\big) = P\big(X_j + n_j^b > \varphi\big) \leq \left(\frac{E[X_j]}{\varphi - n_j^b}\right)^d, \tag{5}$$

where d is the number of rows in the CM Sketch. Substituting (3) and (4) into (5) yields:

$$P_j = P(\hat{n}_j > \varphi) \leq \left[\frac{N\,(1-p)}{\omega\,(\varphi - n_j + n_j p)}\right]^d. \tag{6}$$

Theorem 2: For a DDoS flow f_i, the probability P_{miss}^j of being misclassified as non-DDoS satisfies the following inequality:

$$P_{miss}^j \leq 1 - \sum_{i=0}^{n_j-\varphi-1} \frac{e^{-Np}(Np)^i}{i!} \tag{7}$$

Proof: The process of inserting N data elements into a Bloom Filter is equivalent to N Bernoulli trials. Suppose the false positive probability of the Bloom Filter be p. The probability of k false positives in N trials follows a binomial distribution $B(N,p)$. When N is large, p is small, and Np does not tend to infinity, the binomial distribution can be approximated by a Poisson distribution with $\lambda = Np$. The probability of k false positives is then given by:

$$P(X=k) = \frac{e^{-\lambda}\lambda^k}{k!} \tag{8}$$

The probability of at least k false positives is:

$$\begin{aligned} P(X \geq k) &= 1 - P(X < k) \\ &= 1 - \sum_{i=0}^{k-1} \frac{e^{-\lambda}\lambda^i}{i!} \end{aligned} \tag{9}$$

In the Bloom Filter, the flow f_i is misclassified as non-DDoS if it is misjudged at least $n_j - \varphi$ times. The probability of this event can be equated to the probability of k false positives in N Bernoulli trials, i.e.:

$$P(X \geq k) = P(X \geq n_j - \varphi) \tag{10}$$

where n_j represents the fan-in of flow f_i.

Since the actual error boundary of the Bloom Filter is smaller than p, i.e., when the probability of heads is lower, the probability of at least k heads is also lower. Therefore:

$$P_{miss}^j \leq P(X \geq k) = 1 - \sum_{i=0}^{k-1} \frac{e^{-Np}(Np)^i}{i!} \tag{11}$$

V. The HashGuard

A. The Data Structure

To more accurately track and identify potential DDoS victims, and to reduce the overhead of frequently reporting victim IPs, HashGuard employs a hash table to record destination nodes whose Fan-in exceeds a specific threshold φ. As shown in Fig. 2, HashGuard consists of a Bloom filter, a hash table, and a CM Sketch. Similar to DualSketch, HashGuard uses the Bloom filter to filter packets with identical source and destination IPs, and employs the CM Sketch to record Fan-in values for most destination nodes with relatively small Fan-in.

When a packet arrives at the CM Sketch, if the updated query value in the CM Sketch exceeds the predetermined threshold, HashGuard recirculates the packet to the ingress pipeline, with the aim of storing information related to the packet's destination IP in the hash table. Notably, since a hash table is used to store potential attack victims, numerous DDoS attack packets will only trigger update operations in the hash table, which significantly reduces the frequency of CM Sketch updates. HashGuard's hash table contains C buckets, each comprising two fields: IP_{dst} and v, where v represents the Fan-in value of IP_{dst}. Additionally, when a recirculated packet encounters a collision in the hash table, the destination IP of the packet is sent to the network controller.

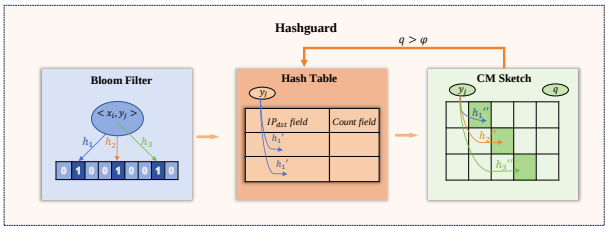


Fig. 2: The famework of HashGuard.

B. Detection Process

Algorithm 2 HashGuard Workflow

```
for each packet pkt do
    Extract source and destination IPs ⟨x_i, y_j⟩ from pkt;
    if ⟨IP_src, IP_dst⟩ not in Bloom Filter then
        Insert ⟨IP_src, IP_dst⟩ into Bloom Filter;
        k = hash(IP_dst);
        if pkt is a recirculated packet then
            if b[k].key == NULL then
                b[k].key = key, b[k].v = φ;
                Forward packet to switch output;
            else
                Send IP_dst to network controller;
            end if
        else
            if b[k].key == IP_dst then
                b[k].v + +;
                Forward packet to output;
            else
                Update CM Sketch and get query value q;
                if q > φ then
                    Recirculate packet;
                end if
            end if
        end if
    else
        Forward packet to output;
    end if
end for
```

The detailed detection process of HashGuard is illustrated in **Algorithm 2**. For each arriving packet, HashGuard first extracts the source and destination IPs $\langle IP_{src}, IP_{dst}\rangle$. Then, it queries the Bloom Filter using $\langle IP_{src}, IP_{dst}\rangle$ as the key. If the query returns false, the Bloom Filter is updated and the packet is forwarded to the CM Sketch; otherwise, the packet is directly forwarded to the switch output port. When a packet reaches the CM Sketch, HashGuard first updates the CM Sketch using IP_{dst} as the key and obtains the query value q. If $q > \varphi$, the packet is recirculated to the ingress pipeline. When a recirculated packet arrives at the hash table, HashGuard inserts the corresponding information of the packet into the hash table. During the hash table processing phase, HashGuard calculates the bucket index k using IP_{dst} as the key. If $b[k].key == key$, the count value in the bucket is updated, and then the original packet is forwarded to the switch output. In this way, most DDoS attack traffic only triggers update operations in the hash table without frequently updating the CM Sketch.

C. The Error Analysis

HashGuard introduces a hash table only to temporarily store the detected DDoS victims and effectively reduce the number of insertions into the CM Sketch. Therefore, the detection accuracy of HashGuard is mainly determined by the Bloom Filter and CM Sketch, meaning that HashGuard's detection error also satisfies **Theorem 1** and **Theorem 2**.

VI. Performance Evaluation

We conduct comprehensive experiments to assess the performance of our proposed DDoS victim detection schemes: DualSketch and HashGuard. In these experiments, we implement the detection schemes using Bmv2. We compare DualSketch and HashGuard against INDDoS [3], a state-of-the-art sketch-based DDoS victsim detection scheme.

A. Experimental Setup

Dataset: The dataset used in the experiments is the MAWI Working Group Traffic Archive Packet traces from the WIDE backbone, consisting of daily traces from the transmission link between WIDE and the upstream ISP on January 1, 2024. Each experiment reads approximately 3×10^8 packets, with an average of about 1.023×10^7 flows.

Performance Metrics: We evaluate the performance of our detection algorithms using four metrics: Recall, Precision, and F1 Score, and Average Relative Error (ARE).

Given a predefined threshold φ, we classify detection outcomes into four categories based on the comparison between the true fan-in count (Truth) and the estimated count (Result):

1) True Positive (N_1): Both Truth and Result exceed the threshold.
2) False Negative (N_2): Truth exceeds the threshold, but Result does not.
3) False Positive (N_3): Result exceeds the threshold, but Truth does not.
4) True Negative: Both Truth and Result are below the threshold .

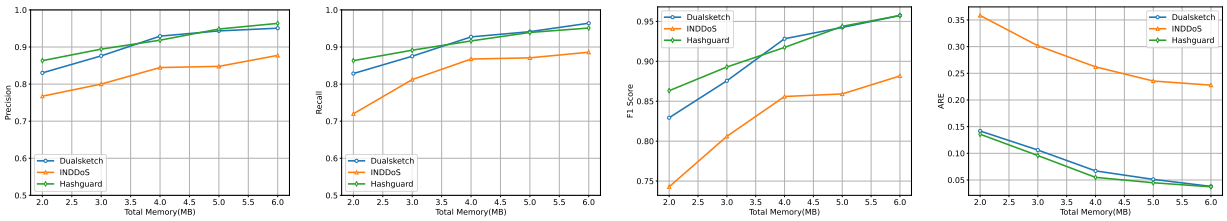
Since our primary concern is with false negatives (missed detections) and false positives (false alarms), we focus on Recall and Precision metrics. Recall measures the proportion of actual DDoS victims that are correctly identified:

$$\text{Recall} = \frac{N_1}{N_1 + N_2} \tag{12}$$

Precision measures the proportion of detected victims that are actual DDoS victims:

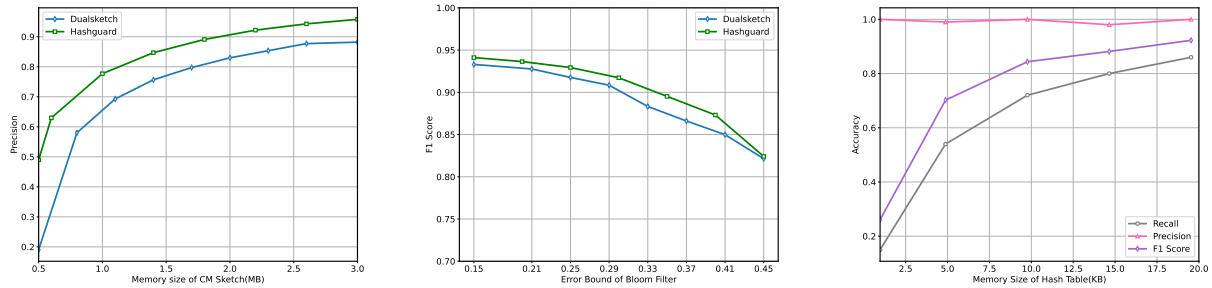
$$\text{Precision} = \frac{N_1}{N_1 + N_3} \tag{13}$$

To balance these two metrics, we use the F1 Score, which is the harmonic mean of Precision and Recall:



(a) Precision (b) Recall (c) F1 Score (d) ARE

Fig. 3: Performance comparison with existing scheme



(a) Precision (b) F1 Score (c) Accuracy

Fig. 4: Performance under different parameters.

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{14}$$

For true positives, we quantify the estimation accuracy using Average Relative Error (ARE):

$$\text{ARE} = \sum_{i=1}^{N_1} \frac{|\text{Truth}_i - \text{Result}_i|}{\text{Truth}_i} \tag{15}$$

Parameters: The number of rows in the CM Sketch for both schemes is set to 3, and the DDoS attack detection threshold φ is set to 112. The bit widths of the two sketches in DualSketch are set to 8 bits and 24 bits respectively.

B. Experimental Results

(1) Comparison with existing detection scheme

Fig. 3 presents the comparison results between Dualsketch, Hashguard, and INDDoS. The experimental results indicate that Dualsketch and Hashguard schemes are superior to the existing INDDoS solution in detecting DDoS victims. As memory increases, the performance metrics of all three schemes improve, but Dualsketch and Hashguard consistently perform better. Under the same memory conditions, the ARE of these two schemes is approximately 0.2 lower than that of INDDoS, indicating higher statistical accuracy. The F1 Score analysis shows that Hashguard performs best with memory below 3.5MB, while Dualsketch is more effective in the 3.5-5MB range; when memory reaches 5-6MB, both schemes achieve an F1 Score of about 0.95, significantly higher than INDDoS. Notably, with only 3MB of memory, the proposed schemes can achieve over 90% F1 Score, demonstrating their ability to efficiently balance precision and recall under limited resource conditions, making them highly suitable for DDoS victim identification in programmable network environments.

(2) The Impact of Different Parameters on Performance

We explored how different parameters affect the performance of DualSketch and Hashguard. In the first experiment, we fixed the Bloom filter error bound and hash table memory at 1×10^{-3} and 20KB, respectively, and assessed the impact of CM Sketch's memory size on detection precision. Fig. 4(a) shows that increasing total memory enhances Precision for both schemes by reducing CM Sketch errors and false positives. At 2.0MB memory, DualSketch achieved a Precision of 0.83, while Hashguard reached 0.91, suggesting that 2-3MB memory balances overhead and accuracy. Hashguard's higher Precision is due to its hash table isolating DDoS victims, which reduces CM Sketch errors for non-DDoS flows.

In the second experiment, we fixed CM Sketch and hash table memory at 2.3MB and 20KB, respectively, and examined Bloom filter error rates. Figure Fig. 4(b) illustrates that higher Bloom filter errors decrease F1 Score. At error bounds of 0.15–0.25, F1 Score remained between 0.92–0.94, indicating few missed detections and tolerance for Bloom filter errors.

VII. Conclusion

This paper presented two novel sketch-based schemes, DualSketch and HashGuard, for efficient DDoS victim detection

in programmable networks. Both schemes address the critical challenge of resource constraints while maintaining high detection accuracy. DualSketch innovatively employs a dual-layer CM Sketch with a carry mechanism to optimize memory usage, while HashGuard combines CM Sketch with hash tables to dynamically track potential victims. Our theoretical analysis provides error bounds that guarantee detection accuracy under limited memory constraints. Experimental results demonstrate that both schemes achieve high precision with minimal memory requirements.

REFERENCES

[1] M. De Vivo, E. Carrasco, G. Isern, and G. O. De Vivo, "A review of port scanning techniques," *ACM SIGCOMM Computer Communication Review*, vol. 29, no. 2, pp. 41–48, 1999.

[2] Y. Wang, S. Wen, Y. Xiang, and W. Zhou, "Modeling the propagation of worms in networks: A survey," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 2, pp. 942–960, 2013.

[3] D. Ding, M. Savi, F. Pederzolli, M. Campanella, and D. Siracusa, "In-network volumetric ddos victim identification using programmable commodity switches," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 1191–1202, 2021.

[4] C. Lapolli, J. Adilson Marques, and L. P. Gaspary, "Offloading real-time ddos attack detection to programmable data planes," in *2019 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*, pp. 19–27, 2019.

[5] A. Santoyo-González, C. Cervelló-Pastor, and D. P. Pezaros, "High-performance, platform-independent ddos detection for iot ecosystems," in *2019 IEEE 44th Conference on Local Computer Networks (LCN)*, pp. 69–75, 2019.

[6] D. Ding, M. Savi, and D. Siracusa, "Tracking normalized network traffic entropy to detect ddos attacks in p4," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 4019–4031, 2022.

[7] R. Wang, Z. Jia, and L. Ju, "An entropy-based distributed ddos detection mechanism in software-defined networking," in *2015 IEEE Trustcom/BigDataSE/ISPA*, vol. 1, pp. 310–317, IEEE, 2015.

[8] F. Musumeci, A. C. Fidanci, F. Paolucci, F. Cugini, and M. Tornatore, "Machine-learning-enabled ddos attacks detection in p4 programmable networks," *Journal of Network and Systems Management*, vol. 30, pp. 1–27, 2022.

[9] M. Dimolianis, A. Pavlidis, and V. Maglaris, "Signature-based traffic classification and mitigation for ddos attacks using programmable network data planes," *IEEE Access*, vol. 9, pp. 113061–113076, 2021.

[10] F. Mecerhed, A. Benabdallah, K. Zeraoulia, and C. Benzaïd, "3d-pm: a ml-powered probabilistic detection of ddos attacks in p4 switches," in *2023 IEEE International Mediterranean Conference on Communications and Networking (MeditCom)*, pp. 80–85, IEEE, 2023.

[11] Y. Cao, H. Jiang, Y. Deng, J. Wu, P. Zhou, and W. Luo, "Detecting and mitigating ddos attacks in sdn using spatial-temporal graph convolutional network," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 3855–3872, 2022.

[12] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.

[13] C. Estan and G. Varghese, "New directions in traffic measurement and accounting," in *Proceedings of the 2002 conference on Applications, technologies, architectures, and protocols for computer communications*, pp. 323–336, 2002.

[14] M. Charikar, K. Chen, and M. Farach-Colton, "Finding frequent items in data streams," *Theoretical Computer Science*, vol. 312, no. 1, pp. 3–15, 2004.

[15] V. Clemens, L.-C. Schulz, M. Gartner, and D. Hausheer, "Ddos detection in p4 using hyperloglog and countmin sketches," in *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*, pp. 1–6, 2023.

[16] Z. Liu, A. Manousis, G. Vorsanger, V. Sekar, and V. Braverman, "One sketch to rule them all: Rethinking network flow monitoring with univmon," in *Proceedings of the 2016 ACM SIGCOMM Conference*, pp. 101–114, 2016.

[17] Y. Zhauniarovich and P. Dodia, "Sorting the garbage: Filtering out drdos amplification traffic in isp networks," in *2019 IEEE Conference on Network Softwarization (NetSoft)*, pp. 142–150, 2019.